

[← Back to Articles](#)

# NVIDIA Kumo Tabular Sets a New Accuracy-Efficiency Frontier for Tabular Prediction

**Enterprise****Article**

Published September 29, 2026

▲ Upvote **66** +60**Jingang Qu**[jingangqu](#)[Follow](#) nvidia**Valter Hudovernik**[valterh-nv](#)[Follow](#) nvidia**Martin Jurkovic**[martinjurkovic](#)[Follow](#) nvidia**Cedric Lorenz**[cedric-lorenz](#)[Follow](#) nvidia**Akihiro Nitta**[akihiironitta-nv](#)[Follow](#) nvidia**Dmitry Gordeev**[dgordeev](#)[Follow](#) nvidia**Federico Lopez**[flopeznv](#)[Follow](#) nvidia**Ramona Bendias**[RBendiasN](#)[Follow](#) nvidia**Gilberto Titericz Jr**[titericz](#)[Follow](#) nvidia**Aleksandar S. Sokolovski**[aleksssokolovski](#)[Follow](#) nvidia**Isabel Hulseman**[ihulseman0220](#)[Follow](#) nvidia**Jure Leskovec**[profjure](#)[Follow](#) nvidia**Matthias Fey**[rusty1s](#)[Follow](#) nvidia

## 🔗 Highlights (TL;DR)

NVIDIA Kumo Tabular, part of the NVIDIA Kumo Structured model collection, is an open foundation model for tabular data now available on [Hugging Face](#). Given a table of labeled rows, it predicts the labels of new rows in a single forward pass, with no training, no tuning, and no feature engineering, for both classification and regression. It was pretrained only on artificial data, comes in three sizes (28M to 215M parameters), runs through our [open-source library](#), and is released under the [OpenMDW-1.1 license](#) for commercial use. It ranks first on the four benchmarks [TabArena](#), [BeyondArena](#), [TALENT](#) and [ScoringBench](#).

- **Model Code:** <https://github.com/NVIDIA/structured-data-models>
- **Model Weights:** <https://huggingface.co/nvidia/Kumo-Tabular>

## 🔗 The Shift to Tabular Foundation Models

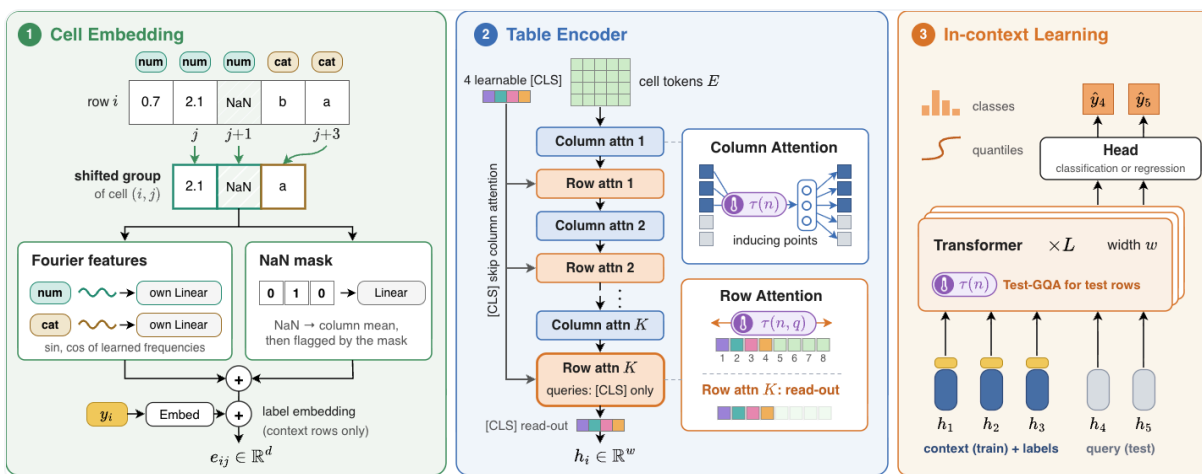
Tabular data is the backbone of enterprise machine learning. Customer records, transactions, sensor logs, claims, and orders all live in tables, and predicting churn, default, demand, or price from them is among the most common machine learning tasks in industry. For two decades, this work has been done with gradient-boosted trees, and it has worked well. But the lifecycle around those models has barely changed. Every new question means collecting labels, engineering features, searching hyperparameters, validating, and deploying a model that knows nothing about tables in general and learns each task from scratch.

Large Language Models showed a different way of working with new tasks. Given a few examples in the prompt, a pretrained model solves the task without updating a single weight. This is *in-context learning*, and it applies to tables just as well as to text: a model pretrained on millions of tables can read a labeled table as its context and predict the labels of new rows directly.

Today, we are releasing **NVIDIA Kumo Tabular** ([GitHub](#), [HuggingFace](#)), an open foundation model for tabular classification and regression. Given a table with labeled rows and the rows you want predictions for, Kumo Tabular returns class probabilities or numeric predictions in a single forward pass.

## 🔗 How Kumo Tabular Works

Kumo Tabular is a Transformer built around the structure of a table, utilizing column, row and in-context attention as introduced in [TabICL](#) and [TabPFN](#). To predict a label it has to do three things: **(1)** understand what each value means within its column, **(2)** understand how the columns of a row interact, and **(3)** relate the context rows with existing labels to the query rows with unknown labels. Kumo Tabular achieves this as follows:



**Cell Embedding:** A group of cells becomes a token. Numerical and categorical values pass through Fourier features, sines and cosines of learned frequencies, with separate weights for each type. Missing values need no imputation and are treated specially. Finally, every token in the context receives a label embedding.

**Row Embedding:** We then turn each row into an embedding by alternating two kinds of attention multiple times. Column attention looks down a single column and learns what a value means in the distribution of its column, *e.g.*, whether a 42 is

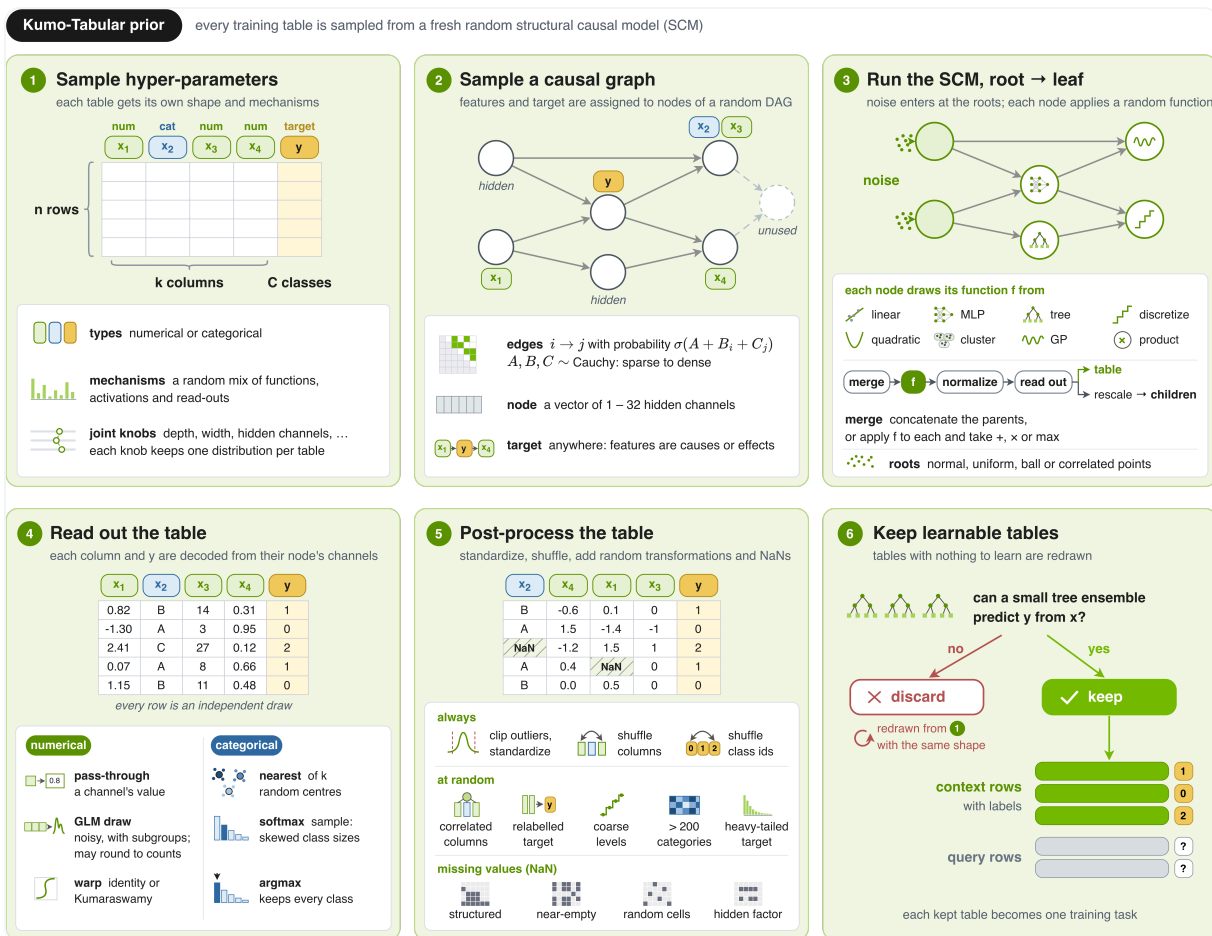
typical or extreme, via induced self-attention. Its cost therefore grows linearly with the number of rows. Row attention looks across the tokens of a single row and learns how features interact, with rotary positions to tell columns apart. Four learnable [CLS] tokens join each row and act as the final readout of a row. After this row compression, the cost of the final stage no longer depends on the number of columns.

**In-context Learning:** A final Transformer operates on the row embeddings. Context rows attend to each other, while query rows attend to context rows only. Each prediction therefore depends only on the context and on the row itself, not on which other rows are scored alongside it. Because the context never looks at the queries, its keys and values are computed once and can be reused for follow-up predictions. Query rows utilize Test-GQA, which shrinks the cache that every prediction reads. A head turns each query row into class probabilities for classification and 999 quantiles for regression, from which a point prediction and an uncertainty estimate follow.

**Length-aware Attention Temperature:** Softmax attention spreads out as the number of keys grows. Attention that is sharp over a few hundred rows can dissolve over tens of thousands, which is exactly the situation when a table at inference is much larger than a typical training table. Kumo Tabular therefore scales every query by a temperature that grows with the logarithm of the number of keys, with a coefficient learned separately for each attention head. The result is attention that stays sharp as tables grow longer or wider.

## 🔗 How Kumo Tabular was Built

Kumo Tabular is pretrained entirely on artificial tables. Each training table is sampled from a *Structural Causal Model (SCM)* in the six steps shown below:



We first draw a configuration for the whole table, from its size and task to its mechanisms and missingness. A random causal graph then links hidden variables, evaluated from root to leaf via randomly drawn functions at every node (e.g., linear maps, small neural networks, trees or Gaussian processes). Some nodes become numerical or categorical columns, one becomes the target, and the rest stay hidden, like the unmeasured causes behind real data. Post-processing correlates groups of columns, clips outliers, and injects missing values, and a quick tree-ensemble check discards any table without a learnable signal. Because the generator is a procedural sampler rather than a trained model, it produces an endless supply of tables, each with a new graph and new mechanisms.

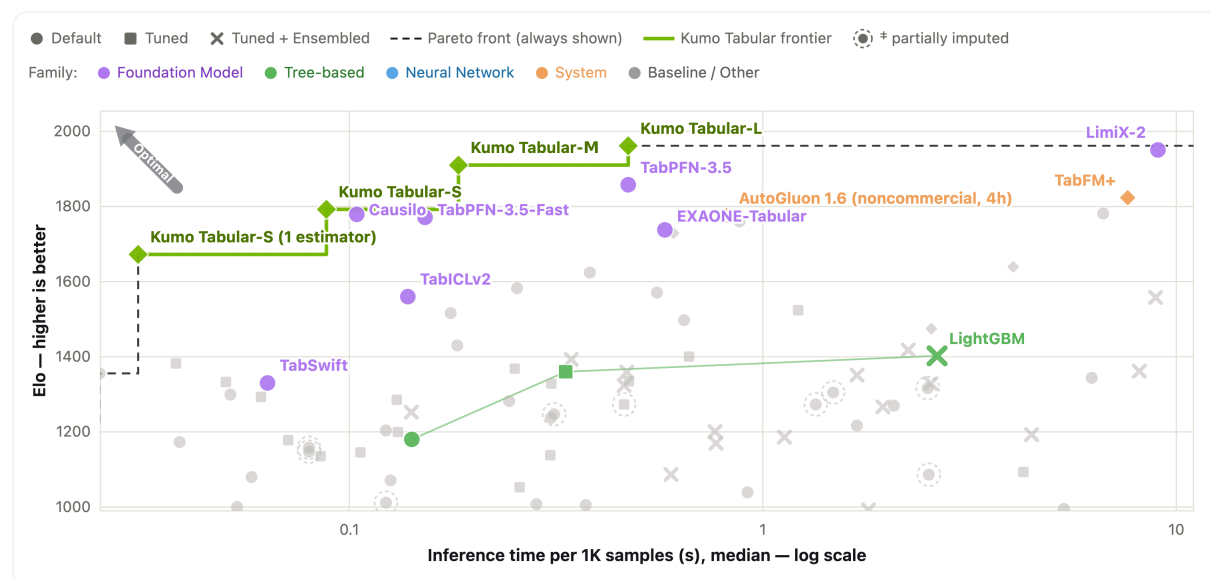
Real-world tables are messy, so we built more of their imperfections into the generator. Values go missing in several patterns, some features are coarsened so that duplicate rows may disagree on their label, some categorical columns carry many levels, and regression targets can be heavy-tailed. A model that has seen millions of such tables learns to handle these imperfections without any cleanup.

On every artificial table, the model sees most of the rows with their labels as context and learns to predict the labels of the remaining rows, with a cross-entropy loss for classification and a quantile loss for regression. Classification and regression are trained as separate models. Similarly to TabICLv2, training runs in three stages. The first and longest stage uses tables of 1,024 rows and up to 100 columns and teaches the model what tables look like. The second stage varies the context from 400 to 10,240 rows, and the third extends it to 60,000 rows, still with up to 100 columns. In total, Kumo Tabular-Small/Medium/Large saw about 35/71/137 million artificial tables.

Our training recipe and artificial data generators will be released soon.

## 🔗 Performance

We ran all three Kumo Tabular sizes with default settings against the full [TabArena](#) leaderboard, spanning tuned gradient-boosted trees, AutoGluon, and the latest tabular foundation models. Kumo Tabular ranks first overall with an ELO of 1950 while running 17 faster than [LimiX-2](#) under a uniform single RTX 6000 Pro evaluation setup. Across all three model sizes, Kumo Tabular establishes a new state-of-the-art on the accuracy-efficiency Pareto front:



We also evaluated Kumo Tabular on [BeyondArena](#), [TALENT](#) and [ScoringBench](#). On BeyondArena, Kumo Tabular reaches an ELO of 1418 with an Improvability score of 7.78%, placing first on the leaderboard. On TALENT, it achieves the top overall ranking across classification accuracy, classification log-loss, and regression RMSE, with average ranks of 6.67, 3.98, and 4.22. On ScoringBench, a benchmark for predictive distributions, Kumo Tabular-Large and Medium rank first and second on average rank.

## Limitations

Kumo Tabular works on numerical and categorical columns only, while text, images, or timestamps can be turned into features via built-in pre-processing recipes. A single forward pass covers up to 10 classes, which the library extends to any number of classes with error-correcting output codes. Accuracy may degrade on tables far beyond the training ranges or when the query rows come from a different distribution than the context rows, so, as with any predictive model, validate accuracy and calibration on your own held-out data before deployment.

## Demo

Kumo Tabular runs via NVIDIA's newly released GPU-native library for [structured-data-models](#). The library downloads the weights from the Hub on first use and provides the preprocessing, ensembling, and many-class handling used in our evaluations. The code below is all it takes to go from a `pandas.DataFrame` to a prediction:

```
import sdm # structured-data-models

# Tensorize tabular data:
table = sdm.TableTensor.from_pandas(pd.load_csv(...), device="cuda")
na_mask = table["target"].isnan()
```

```
model = sdm.models.KumoTabular(device="cuda")
pred = model(
    # In-context examples (features/targets):
    x_context=table[~na_mask].drop_columns("target"),
    y_context=table[~na_mask, "target"],
    # Prediction examples (features):
    x_query=table[na_mask].drop_column("target"),
)
```

## 🔗 Start Building with Kumo Tabular

Kumo Tabular is released under the [OpenMDW License Agreement, version 1.1](#). NVIDIA believes Trustworthy AI is a shared responsibility, and we have established policies and practices to enable development for a wide array of AI applications. When downloaded or used in accordance with our terms of service, developers should work with their supporting model team to ensure this model meets requirements for the relevant industry and use case and addresses unforeseen product misuse. Please report model quality, risk, security vulnerabilities, or NVIDIA AI concerns [here](#).

- **Model Code:** <https://github.com/NVIDIA/structured-data-models>
- **Model Weights:** <https://huggingface.co/nvidia/Kumo-Tabular>

## 🔗 Acknowledgements

We thank [David Holzmüller](#) for contributing significant ideas and ablations to Kumo Tabular. We thank [Vignesh Kothapalli](#) for his help on Kumo Tabular during his internship.

## Models mentioned in this article 1

 [nvidia/Kumo-Tabular](#)

Updated about 16 hours ago •  99

## More from this author



### How to Use NVIDIA Warp and MjWarp to Accelerate Robotics Simulation and Learning Workflows

 [nvidia](#)

 53 September 23, 2026



**NVIDIA**



## **\*\*Know Who Spoke When: Build Real-Time, Multi-Speaker AI with NVIDIA Nemotron 3 Diarization\*\***

 nvidia

△ 66 September 23, 2026

### Community

 **Dhanjeerider** 3 days ago



Good



 Reply

 **matthias-matthias** 2 days ago



Looks great. Thanks! FYI, currently, in the sentence "Kumo Tabular is released under the OpenMDW License Agreement, version 1.1." the link to the license is broken.

 1 reply · 

 **rusty1s** Article author about 20 hours ago



Fixed, thanks!



Reply in thread

 **JoZZa88** about 3 hours ago



Fascinating. I ran some small benchmarks and comparisons on my quarto page:

<https://cjosh88.github.io/ai-portfolio/experiments/2026-10-kumo-tabular/>



 Reply

Edit

Preview

Start discussing this article

 Tap or paste here to upload images

Comment

Sign up or log in to comment

 System theme

Company

TOS

Privacy

About

Careers

Website

Models

Datasets

Spaces

Pricing

Docs

